Support de cours : gérer un projet d'équipe avec GitHub (Scrum)

Exemple concret : le projet Billetterie, octobre 2026. À réutiliser pour tes prochains projets.

1. Le vocabulaire

Terme	Ce que c'est	Dans Billetterie
Dépôt (repository)	Le dossier du projet hébergé sur GitHub, avec tout son historique	massepaul19/Billeterie
Branche	Une ligne de travail parallèle, isolée des autres	main, develop, feature/auth-jwt-balla
Commit	Une sauvegarde de modifications, avec un message	feat(auth): ajout du filtre JWT
Pull request (PR)	Une demande de fusionner une branche dans une autre, avec relecture	feature/auth-jwt-balla vers develop
Issue	Une tâche ou un bug, avec sa discussion	A1 - Auth/JWT (#1)
Jalon (milestone)	Un groupe d'issues avec une échéance	Sprint 1, Sprint 2, Sprint 3
Étiquette (label)	Un mot coloré pour classer les issues	back-a, front-d, bug-prod
Projet (GitHub Projects)	Le tableau de suivi	Billetterie - Suivi (n°7)
Statut	La colonne d'une carte dans le tableau	Todo, In Progress, En revue, Done
Tag	Un repère posé sur une version	v0.1, v0.2, v1.0

Correspondance avec Scrum

Scrum	GitHub
Backlog	Les issues
Sprint	Un jalon
Tableau de sprint	La vue Board du projet, filtrée sur le jalon actif
Incrément livré	Un tag (v0.1, v0.2, v1.0)
Définition de terminé	La liste de la règle 8 de DECOUPAGE_TRAVAIL.md

2. Ce qui est en place sur Billetterie

- Un dépôt privé avec deux branches : main (versions stables) et develop (intégration).
- 9 étiquettes : une par rôle (back-a, back-b, front-c, front-d, paulo) et api-change, config, bug-prod, option.
- 3 jalons : Sprint 1 (J3-J7), Sprint 2 (J8-J12), Sprint 3 (J13-J14).
- Un projet « Billetterie - Suivi » avec 4 statuts : Todo, In Progress, En revue, Done.
- 55 issues : 26 au Sprint 1, 20 au Sprint 2, 9 au Sprint 3.
- Une vue Board « Sprint en cours », filtrée sur le jalon du Sprint 1.

3. Le cycle de vie d'une tâche

Exemple : Balla prend l'issue #1 (A1 - Auth/JWT).

1. **Prendre la tâche.** On glisse la carte de Todo vers In Progress dans le tableau, et on s'assigne l'issue.
2. **Créer sa branche** depuis develop à jour :

```
git switch develop
git pull
git switch -c feature/auth-jwt-balla
```

3. **Coder par petits commits** : feat(module): ..., fix(module): ..., test(module): ..., docs: ...
4. **Pousser la branche** :

```
git push -u origin feature/auth-jwt-balla
```

5. **Ouvrir la PR vers develop** (moins de 300 lignes, une seule fonctionnalité) :

```
gh pr create --base develop --title 'feat(auth): connexion JWT' --body 'Closes #1'
```

6. **Relecture.** La carte passe en En revue. Un coéquipier du même domaine relit, puis l'intégrateur merge.
7. **Merge, puis Done.** Après le merge, l'issue est fermée et la carte passe à Done.
8. **Nettoyer.** On supprime la branche fusionnée.

Attention : Closes #1 ne ferme pas toujours l'issue

GitHub ne ferme automatiquement l'issue que si la PR est fusionnée dans la **branche par défaut** du dépôt. Aujourd'hui c'est **main**, alors que toutes les PR visent **develop**. Les issues ne se fermeraient donc pas toutes seules. Deux solutions :

- Mettre **develop** comme branche par défaut (le plus simple, fréquent quand on travaille avec une branche d'intégration) :

```
gh repo edit massepaul19/Billeterie --default-branch develop
```

- Ou fermer chaque issue à la main après le merge.

Faut-il cocher des cases ?

Il y a trois sortes de « cases », à ne pas confondre :

Case	À cocher ?
Le rond vert d'une issue (ouverte), qui devient violet (fermée)	Non, il change quand l'issue se ferme
Les cases - [ ] écrites dans le texte d'une issue	Oui, si on les utilise comme sous-étapes ; GitHub compte l'avancement
Les cases du TODO.md d'un module	Oui : c'est l'une des 5 conditions de la définition de terminé, à cocher dans la PR

Une tâche est **Done** seulement quand les 5 conditions de la règle 8 sont réunies : contrat API respecté, tests qui passent, aucun secret dans Git, case du TODO.md cochée, PR relue et liée à l'issue.

Le projet propose aussi des automatisations (menu **Workflows**), par exemple passer en Done à la fermeture d'une issue. Elles sont à vérifier dans ce menu. Le passage à In Progress se fait à la main, en glissant la carte.

4. Organiser les branches

```
main        versions stables seulement (tags v0.1, v0.2, v1.0)
  +-- develop       integration de tout le travail
        |-- feature/auth-jwt-balla
        |-- feature/recherche-voyages-patricia
        +-- feature/auth-abdias
```

- Chacun crée **une branche par tâche** depuis develop, et ne pousse que sur elle.
- L'intégrateur fusionne les PR dans develop, teste, puis fusionne develop dans main à chaque version.
- **Pas de branches front-end et back-end partagées.** Les dossiers frontend/ et backend/ séparent déjà le code. Une branche longue partagée par deux personnes ajoute un niveau de fusion, retarde l'intégration et mélange les travaux. Une branche par tâche reste plus simple.
- **L'intégrateur sur develop** : possible pour un petit changement de fichier partagé (pom.xml, schema.sql, API.md), à condition de faire git pull juste avant git push. Pour tout ce qui dépasse quelques lignes, une courte branche garde une trace claire.
- Chaque matin : git switch develop puis git pull, avant de commencer.

5. Inviter des collaborateurs

Il faut leur nom d'utilisateur GitHub (celui de l'adresse github.com/nom).

```
gh api users/NOM --jq '.login, .name'
gh api -X PUT repos/massepaul19/Billeterie/collaborators/NOM -f permission=push
gh api repos/massepaul19/Billeterie/invitations --jq '.[] | .invitee.login'
```

- La première commande vérifie que le compte existe et que c'est bien la bonne personne.
- La deuxième envoie l'invitation. **L'invité doit l'accepter** (e-mail ou notification GitHub), sinon il ne voit pas le dépôt privé.
- La troisième liste les invitations en attente.

Niveaux de permission : read (lire), triage (gérer les issues), push (écrire des branches et ouvrir des PR, c'est le bon niveau pour l'équipe), maintain, admin.

Une fois l'invitation acceptée, on peut leur assigner leurs issues, dans le tableau ou avec gh issue edit NUMERO --add-assignee NOM.

6. Le tableau : que faire à chaque étape

Situation	Action
Une tâche démarre	Carte vers In Progress, s'assigner l'issue, créer la branche
Le code est prêt	Ouvrir la PR avec Closes #numéro, carte vers En revue
La PR est relue et fusionnée	L'issue se ferme, la carte passe à Done
Une anomalie est trouvée	Nouvelle issue avec l'étiquette bug-prod : étapes, résultat obtenu, résultat attendu, capture d'écran
Il faut changer le contrat d'API	Issue api-change, on ne modifie jamais API.md soi-même

Fin d'un sprint

1. Intégration : fusionner le travail dans develop, brancher le front sur le vrai back, corriger les écarts.
2. Démonstration, puis rétrospective.

3 Fusionner develop dans main (par une PR), puis poser le tag :

```
gh release create v0.1 --target main --title 'v0.1' --notes 'Sprint 1 : catalogue de voyages'
```

4 Les issues non terminées : les déplacer vers le jalon suivant, ou les fermer avec une explication.

5 Fermer le jalon (le numéro est visible avec gh api repos/massepaul19/Billeterie/milestones --jq '.[] | .number, .title') :

```
gh api -X PATCH repos/massepaul19/Billeterie/milestones/1 -f state=closed
```

6 Dans la vue **Sprint en cours**, remplacer le nom du jalon dans le filtre par celui du sprint suivant, puis enregistrer.

7 Planification du sprint suivant : relire les tâches, poser les questions, s'engager.

7. Aide-mémoire gh

Besoin	Commande
Se connecter	gh auth login
Ajouter le droit sur les projets	gh auth refresh -s project
Voir l'état de la connexion	gh auth status
Créer une étiquette	gh label create NOM -R dépôt --color HEX --description '...' --force
Créer un jalon	gh api repos/dépôt/milestones -f title='...'
Créer un projet	gh project create --owner NOM --title '...'
Relier un projet au dépôt	gh project link NUMERO --owner NOM --repo dépôt
Créer une issue	gh issue create -R dépôt --title '...' --label x --milestone '...' --project '...' --body '...'
Lister les issues	gh issue list -R dépôt --limit 100
Lire une issue	gh issue view NUMERO -R dépôt
Lister les cartes du projet	gh project item-list NUMERO --owner NOM
Créer une PR	gh pr create --base develop --title '...' --body 'Closes #N'
Lister les PR	gh pr list
Voir la visibilité du dépôt	gh repo view dépôt --json visibility

8. Les pièges rencontrés

- **Un paquet snap non officiel** : celui de gh venait d'un particulier. On a installé la version officielle depuis le dépôt de GitHub. Toujours regarder l'éditeur du paquet.
- **Une chaîne de commandes avec &&** s'arrête à la première erreur : l'erreur d'un autre dépôt apt a empêché l'installation de gh, qu'il a fallu lancer à part.
- **Le collage de gros blocs** peut brouiller l'affichage du terminal sans abîmer les commandes. On vérifie avec gh issue view. Pour les très gros blocs, mieux vaut les écrire dans un fichier .sh, puis les lancer avec bash.
- **Relancer un bloc crée des doublons** : les jalons et les issues ne sont pas protégés comme les étiquettes. On lance une seule fois, puis on compte avec gh issue list.
- **grep distingue majuscules et minuscules** : utiliser grep -i, et exclure les dossiers lourds avec --exclude-dir=build.

- **Le dépôt est privé** : la protection de branches est réservée aux comptes payants sur un dépôt privé. À vérifier en essayant, sinon la règle reste une convention d'équipe.

9. Où en est Billetterie

Fait : dépôt, branches main et develop, étiquettes, jalons, tableau n°7, 55 issues, vue Sprint en cours, base SQLite complète (17 tables).

À faire :

- Inviter Balla, Patricia (compte patriciasydney, invitation à envoyer et à faire accepter), Abdias et Bouba.
- Assigner les issues à chacun une fois les invitations acceptées.
- Décider de la branche par défaut (develop, pour que Closes # fonctionne).
- Enregistrer dans Git la correction de DECOUPAGE_TRAVAIL.md (ligne 307), puis pousser.
- Protéger main et develop si le compte le permet.
- Annoncer à l'équipe les règles : une branche par tâche, PR vers develop, pas de push direct.